

DOCUMENTATION

Prediction Marketplace Security Architecture Audit Report

A comprehensive review of security controls, formal verification models, database hardening strategies, and system jailbreak protections.

Prepared For: Prediction Marketplace Audit Handover

Version: v8.1 (Base Sepolia & Hoodi Network)

Author: Protocol Lead Security Architect & Google Deepmind team

1. Executive Security Summary

The SX Prediction Marketplace is a decentralized, high-concurrency prediction market platform running on EVM-compatible networks (Base Sepolia and Hoodi Network). Given the financial exposure involved in market resolutions and secondary market position trading, the system enforces a defense-in-depth model spanning three distinct layers: the Smart Contract layer, the Backend REST API layer, and the Database Storage layer.

This report documents the security posture of all three layers, highlighting formal verification properties, active threat mitigation mechanisms (such as rate limiters and input inspection firewalls), and structural database safeguards designed to ensure system resilience and audit readiness.

2. Smart Contract Layer Defenses

All core protocol smart contracts are designed under upgradeable proxy architectures (using OpenZeppelin's ERC1967 pattern) to allow patch deployment under emergency state lockouts. Access control is strictly segregated using a centralized role manager contract (`SXAccessControlUpgradeable`).

2.1 Access Control Role Segregation

Role Name	Assigned Account	Permissions / Responsibilities
<code>DEFAULT_ADMIN_ROLE</code>	Admin Multisig	Manage protocol configurations, grant/revoke roles, execute contract upgrades.

Role Name	Assigned Account	Permissions / Responsibilities
OPERATOR_ROLE	Backend Worker	Enqueue system maintenance, trigger daily accuracy updates, pause/unpause contracts.
RESOLVER_ROLE	Oracle / Resolver	Resolve active prediction markets (triggering payout estimations).
MARKET_CREATOR_ROLE	Authorized Creators	Create new prediction markets on the Factory.

2.2 Formal Verification Specs & Mathematical Invariants

To mathematically guarantee the safety of smart contract operations, the contracts were audited under the Certora Prover bounded model checker and Foundry invariant fuzzer tests. The following critical properties have been verified:

- **Solvency Invariant:** The total pool balance must exactly equal the sum of all YES and NO stakes (`totalPool = yesPool + noPool`) at all times, preventing math manipulation.
- **No Negative Balances:** A user's unified account balance in `SXUUpgradeable` can never drop below zero under any deposit, withdraw, or staking sequence.
- **Resolution Monotonicity:** A resolved prediction market can never be unresolved or modified. The winning outcome must remain immutable post-resolution.

3. Backend REST API Hardening

The backend application, written in Node.js (TypeScript), acts as the primary orchestrator between frontend requests, database state, and contract event

synchronization. It incorporates several active defenses to block malicious traffic and prevent abuse.

3.1 Input Payload Security Scanning (WAF Middleware)

A Web Application Firewall (WAF) style middleware (`securityScanner`) intercepts all incoming request bodies and query parameters. It scans parameters against a set of signatures designed to block instruction injections, system prompt bypasses, and SQL injection threats.

```
// Suspicious patterns scanned on all inputs

const suspiciousPatterns = [

  "ignore previous instructions",

  "bypass",

  "admin access",

  "override role",

  "jailbreak",

  "system prompt",

  "select * from",

  "drop table"

];
```

When a pattern matches, the request is immediately aborted with a `400 Bad Request`. The intrusion details (wallet address, IP address, exact payload, severity, and timestamp) are written to the database for audit review.

3.2 API Rate Limiting Configuration

To mitigate Denial of Service (DoS) attacks and brute-force attempts on sensitive routes (such as authentication and transaction submissions), the API implements a Redis-backed sliding window rate limiter.

API Endpoint Prefix	Request Limit	Time Window	Behavior on Limit Breach
/api/auth/*	5 requests	15 minutes	Blocks requests, returns HTTP 429, logs warning.
/api/* (Standard)	100 requests	15 minutes	Blocks requests, returns HTTP 429.
/api/health	Unlimited	N/A	No rate limiting (to support constant uptime monitoring).

3.3 Multi-Factor Authentication (MFA/TOTP)

Administrator functions (such as resolving markets manually or distributing leaderboard rewards) require a valid JWT token combined with a Time-Based One-Time Password (TOTP) seed generated via Google Authenticator. This protects the protocol from session hijacking even if the operator's private keys or login credentials are compromised.

4. Database Storage Layer Hardening

The database layer uses PostgreSQL to house platform configurations, indexed contract logs, user metadata, and security log entries. The following mechanisms protect database integrity:

4.1 Read-Only Role Isolation

A specific database user `marker` is created with strictly limited read-only permissions (`SELECT` only) to query specific tables. The user is blocked from modifying any data (no `INSERT`, `UPDATE`, `DELETE`, or DDL operations) and cannot access security logs or raw password hashes.

```
-- Read-Only provisioning script snippet

CREATE USER marker WITH PASSWORD 'marker_readonly_pass';

GRANT CONNECT ON DATABASE sx_prediction_db TO marker;

GRANT USAGE ON SCHEMA public TO marker;

GRANT SELECT ON ALL TABLES IN SCHEMA public TO marker;

ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT SELECT ON TABLES TO
marker;

REVOKE SELECT ON TABLE "SecurityLog" FROM marker;
```

4.2 Backup and Recovery Policy

To guarantee business continuity and recovery from catastrophic infrastructure failure, backups are fully automated.

- **Mechanism:** PostgreSQL `pg_dump` is executed inside the Docker container to capture consistent hot backups without interrupting API services.
- **Retention Schedule:** Daily backups are saved as compressed SQL text files. The system automatically executes a retention script to delete snapshots older than 7 days, maintaining a sliding window of recovery points.
- **Security:** Backups are stored in a root-restricted host directory

`(/var/backups/postgres/)` which prevents unauthorized reading or network exposure of database dumps.

4.3 Data Sanitization & ORM Security

All database queries are executed using the Prisma ORM. Prisma automatically parameterizes SQL queries, entirely eliminating the risk of SQL injection through input values. In addition, request payloads are strictly validated using `zod` schemas before processing, guaranteeing correct types, ranges, and string bounds.

5. Handover Checklist & Audit Verification

Auditors can verify the security controls outlined in this documentation by inspecting the following workspace files:

- **Smart Contract Verification Specs:** Located in `smartcontract/certora/formal-verification/*.spec`
- **Security Scanner Implementation:** Located in `backend/src/middleware/security.ts`
- **Database Read-Only Configuration:** Located in `backend/src/database/init-readonly.sql`
- **Automated Backup Script:** Located in `backend/scripts/backup.sh`